Mayo Clinic

Digital Product Engineering — Test 6

Search and Ranking System

Coding Assessment Narrative

Candidate: Hameem Mahdi M.S.E., Ph.D.

Implementation Language: Python 3 (standard library only)
Source File: search_and_ranking_system.py
Datasets: 7 provided files • 10 tasks • 10 output files

Submission · March 9, 2026

1. Executive Summary

This document presents a complete implementation of the Digital Product Engineering Test 6 coding assessment. The solution delivers all ten required components of a simplified search and ranking system, built to process synthetic healthcare content and search log datasets.

The implementation is contained in a single Python file (search_and_ranking_system.py) that uses only the Python standard library—no external dependencies are required. Each task is implemented as a standalone function, and all tasks produce deterministic, reproducible outputs across runs. Tasks can be executed individually or collectively via command-line arguments.

2. Approach and Design Principles

Zero External Dependencies: The entire solution uses only Python's standard library (csv, json, math, collections, os, argparse, sys). This ensures portability and eliminates environment setup friction—no virtual environment or package installation is needed.

Single-File Architecture: All ten tasks reside in one file organized into clear sections: data loaders, algorithm implementations, output helpers, and task runners. This makes the codebase easy to navigate and review.

Deterministic Outputs: Every function sorts its results using stable, predictable criteria (alphabetical for indices, descending by score for rankings). Two consecutive runs produce byte-identical output files, verified by diff.

No Hardcoded Results: All outputs are computed at runtime from the provided datasets. No values are pre-computed or embedded in the source code.

Modular and Testable: Each task is implemented as a pure function that takes data in and returns results. Task runner functions handle I/O. An argparse-based CLI allows running any subset of tasks independently.

Appropriate Output Formats: Tabular ranked lists are written as CSV files (easy to sort, filter, import). Key-value mappings and nested structures are written as JSON files (preserves hierarchy and readability).

3. Datasets Overview

The assessment package includes seven datasets that simulate a production search environment with healthcare content:

Dataset	Size	Columns	Description
documents.csv	200 documents	doc_id, title, content, freshness, tag	Core document corpus with healthcare titles, text content,

			freshness scores (0–1), and category tags (e.g., cardiology, pulmonology).
ranking_signals.csv	200 documents	doc_id, bm25, freshness, ctr	Precomputed ranking features for each document, used in hybrid ranking.
query_logs.csv	300 log entries	query, clicked_doc	Search logs recording which document a user clicked for a given query.
query_autocomplete.txt	15 queries	plain text, one per line	Historical search queries used to build prefix-based autocomplete suggestions.
document_embeddings.csv	50 documents	doc_id, embedding	3-dimensional vector embeddings for a subset of documents, used for similarity search.
query_synonyms.csv	5 mappings	query_term, rewrite	Synonym pairs mapping colloquial health queries to clinical terms.
rerank_features.csv	5 documents	doc_id, relevance, ctr, authority, freshness	Multi-signal feature vectors for candidate documents in the reranking pipeline.

4. Task-by-Task Implementation

4.1 Task 1 – Build an Inverted Index

Spec: Tokenize document content and build a mapping of tokens to document IDs.

Dataset: documents.csv

Output: task1_inverted_index.json

Approach: The build_inverted_index() function iterates over all 200 documents, tokenizes each document's content field by lowercasing and splitting on whitespace, and maps each unique token to the set of document IDs that contain it. The resulting index is sorted alphabetically by token, and each posting list is sorted numerically by document ID, ensuring deterministic output.

Result: 70 unique terms indexed across 200 documents.

4.2 Task 2 – Compute Term Frequency (TF)

Spec: Implement TF scoring for documents given a query.

Dataset: documents.csv

Output: task2_tf_scores.json

Formula: `TF(t, d) = count(t in d) / total_tokens(d)`

Approach: The compute_tf() function computes term frequency for each query term in every document. TF is calculated as the raw count of the term in the document divided by the total number of tokens in that document. Only documents with at least one non-zero TF score are included in the output.

Result: 46 documents have non-zero TF scores for the test query "diabetes diet."

4.3 Task 3 – Implement BM25 Ranking

Spec: Calculate ranking scores for documents using the BM25 formula.

Dataset: documents.csv

Output: task3_bm25_ranking.csv

Formula:
```
BM25(d, q) = Σ IDF(t) × [tf × (k1+1)] / [tf + k1 × (1 – b + b × dl/avgdl)]
IDF(t) = ln((N – df + 0.5) / (df + 0.5) + 1)
```

Approach: The compute_bm25() function implements the standard BM25 scoring algorithm with parameters k1=1.5 and b=0.75. For each query term, it computes the inverse document frequency (IDF) using the non-negative variant, then combines it with the normalized term frequency accounting for document length relative to the average. All 200 documents are scored and sorted in descending order.

Result: Top 8 documents score 4.7136 (containing both "diabetes" and "diet"), consistent with TF results from Task 2.

4.4 Task 4 – Hybrid Ranking

Spec: Combine multiple signals (BM25, freshness, CTR) into a final ranking score.

Dataset: ranking_signals.csv

Output: task4_hybrid_ranking.csv

Formula: `hybrid_score = 0.5 × BM25 + 0.3 × freshness + 0.2 × CTR`

Approach: The hybrid_ranking() function reads precomputed BM25, freshness, and CTR signals from ranking_signals.csv and combines them using a weighted linear combination. The weights (0.5 for BM25, 0.3 for freshness, 0.2 for CTR) reflect the typical importance hierarchy in search: relevance first, then recency, then engagement.

Result: 200 documents ranked. Doc 40 leads with a hybrid score of 1.351.

4.5 Task 5 – Query Autocomplete

Spec: Implement prefix-based autocomplete using historical queries.

Dataset: query_autocomplete.txt

Output: task5_autocomplete.json

Approach: The autocomplete() function performs case-insensitive prefix matching against the 15 historical queries in query_autocomplete.txt. For each test prefix, it returns all matching queries sorted alphabetically. Three prefixes are tested: "heart" (5 matches), "diabetes" (3 matches), and "lung" (2 matches).

Result: 10 total autocomplete suggestions across 3 test prefixes.

4.6 Task 6 – Search Log Analytics

Spec: Compute query frequency, document click frequency, and query CTR.

Dataset: query_logs.csv

Output: task6_analytics.json

Formula: `CTR(q) = unique_docs_clicked(q) / total_impressions(q)`

Approach: The search_log_analytics() function processes 300 search log entries to compute three metrics: (1) query frequency—how many times each query was searched, (2) document click frequency—how many times each document was clicked across all queries, and (3) query CTR—the ratio of unique documents clicked to total impressions for each query. All metrics are sorted in descending order.

Result: 11 unique queries, 154 unique clicked documents. Top query: "diabetes diet" with 39 occurrences.

4.7 Task 7 – Personalization Boost

Spec: Increase ranking scores when document tags match a user condition.

Dataset: documents.csv + ranking_signals.csv

Output: task7_personalized_ranking.csv

Formula: `boosted_score = hybrid_score × 1.2 (if tag matches user preference)`

Approach: The personalization_boost() function first computes the hybrid ranking from Task 4, then applies a 1.2× multiplicative boost to any document whose category tag matches the user's preferred tag. For the test case, the user tag is "cardiology." This causes cardiology-tagged documents to rise in the rankings while non-matching documents retain their original scores.

Result: Doc 195 rises to #1 (score 1.4724) after boosting, displacing Doc 40 (1.351, unchanged—not tagged cardiology).

4.8 Task 8 – Vector Similarity Search

Spec: Compute cosine similarity between a query embedding and document embeddings.

Dataset: document_embeddings.csv

Output: task8_vector_similarity.csv

Formula: `cosine_sim(a, b) = (a · b) / (||a|| × ||b||)`

Approach: The cosine_similarity() function computes the standard cosine similarity metric: the dot product of two vectors divided by the product of their magnitudes. The vector_similarity_search() function applies this to all 50 document embeddings (3-dimensional vectors) against a test query embedding of [0.5, 0.5, 0.8]. Results are sorted by similarity in descending order.

Result: All 50 documents ranked. Similarity scores range from 0.6607 to 0.9974 (Doc 29).

4.9 Task 9 – Query Rewriting

Spec: Rewrite queries using the synonym mappings provided.

Dataset: query_synonyms.csv

Output: task9_query_rewrites.json

Approach: The rewrite_query() function performs exact match lookup against the 5 synonym mappings in query_synonyms.csv. If a query matches a synonym key (case-insensitive), it is replaced with the mapped clinical term. If no match is found, the original query passes through

unchanged. Six test queries are processed, including one ("diabetes diet") that has no synonym and remains unmodified.

Result: 5 of 6 test queries rewritten: "heart doctor" → "cardiologist," "kid fever" → "pediatric fever," "lung doctor" → "pulmonologist," "blood sugar" → "glucose," "diabetes food" → "diabetes diet."

4.10 Task 10 – Reranking Pipeline

Spec: Apply a reranking formula using relevance, CTR, authority, and freshness features.

Dataset: rerank_features.csv

Output: task10_reranked.csv

Formula: `rerank_score = 0.4 × relevance + 0.3 × CTR + 0.2 × authority + 0.1 × freshness`

Approach: The rerank() function applies a weighted linear combination of four feature signals from rerank_features.csv. The weights prioritize relevance (0.4) and CTR (0.3), followed by authority (0.2) and freshness (0.1). This mirrors a typical production reranking layer where multiple quality signals are fused into a single final score.

Result: 5 candidate documents reranked. Doc 1 leads with a score of 0.745.

5. Output Summary

All ten tasks produce output files in the output/ directory:

Task	Output File	Format	Contents
1	task1_inverted_index.json	JSON	70 terms → sorted doc ID lists
2	task2_tf_scores.json	JSON	46 docs with per-term TF scores
3	task3_bm25_ranking.csv	CSV	200 docs ranked by BM25 score
4	task4_hybrid_ranking.csv	CSV	200 docs ranked by hybrid score
5	task5_autocomplete.json	JSON	3 prefixes → 10 completions
6	task6_analytics.json	JSON	Query freq, click freq, CTR
7	task7_personalized_ranking.csv	CSV	200 docs with

			personalized boost
8	task8_vector_similarity.csv	CSV	50 docs ranked by cosine similarity
9	task9_query_rewrites.json	JSON	6 queries → rewritten forms
10	task10_reranked.csv	CSV	5 docs reranked by weighted features

6. How to Run

Prerequisites: Python 3.7 or later. No external packages required.

Run all tasks:

```
cd "Test 6/engineering_test_6"
python search_and_ranking_system.py
```

Run a single task (e.g., Task 3):

```
python search_and_ranking_system.py 3
```

Run multiple tasks (e.g., Tasks 1, 5, 10):

```
python search_and_ranking_system.py 1 5 10
```

7. Conclusion

This implementation delivers all ten components of the search and ranking system as specified in the assessment. The solution demonstrates:

- Correct implementation of core search algorithms (inverted index, TF, BM25)
- Signal fusion techniques (hybrid ranking, reranking pipeline)
- User-facing search features (autocomplete, query rewriting)
- Analytics capabilities (search log analysis with frequency and CTR metrics)
- Personalization through tag-based score boosting
- Vector search using cosine similarity over document embeddings
- Clean, modular code with deterministic and reproducible outputs

All outputs have been verified for correctness through manual spot-checks, cross-task consistency validation, and determinism testing across multiple runs.